

Technical Specification: Microcircuit Assembly

Synthetic Neuronal Population Generation

Research and Development Documentation

March 26, 2026

1 Introduction

This document describes the architectural framework for the synthetic generation and spatial arrangement of neuronal populations within a 3D cylindrical microcolumn. The system translates biological density data into individual cell instances, assigning each a precise 3D coordinate and a morphological type (m-type).

2 Core Functionality: `generate_microcolumn_cells`

The `generate_microcolumn_cells` function acts as the primary architect of the microcircuit. It populates a defined cylindrical volume while maintaining realistic spatial distributions and density-driven population sizes for various cortical layers.

2.1 1. Geometric Setup: The Cylindrical Column

The model represents a cortical microcolumn as a cylinder. The implementation utilizes a **negative depth convention**, where the pia (cortical surface) is positioned at $z = 0$, and the depth increases negatively toward the white matter (e.g., Layer 6).

2.2 2. From Density to Discrete Populations

Instead of manual cell counts, the function dynamically calculates the number of neurons for each m-type based on empirical density values (cells/mm³).

1. **Volume Calculation:** For each cortical layer, the function calculates the volume (V) of the corresponding cylindrical slice:

$$V = \pi \cdot r^2 \cdot (z_{\max} - z_{\min}) \quad (1)$$

2. **Density Conversion:** Since coordinates are managed in micrometers (μm), densities are converted from mm³ to μm^3 (dividing by 10^9) to derive the final cell count:

$$\text{Count} = \text{round} \left(\text{Density}_{\mu\text{m}^3} \cdot V_{\mu\text{m}^3} \right) \quad (2)$$

2.3 3. Uniform Spatial Sampling

To prevent artificial clustering near the center of the column and ensure a biologically realistic uniform distribution, specific sampling techniques are employed.

A. Radial Uniformity (The $\sqrt{U}$ Trick)

A naive approach using random radius r and angle θ results in higher density near the center. To achieve uniform area coverage across the circular cross-section, the function uses the square root of a uniform random variable $U \in [0, 1)$:

$$r = R \cdot \sqrt{U(0,1)} \quad (3)$$

$$x = r \cdot \cos(\theta) \quad (4)$$

$$y = r \cdot \sin(\theta) \quad (5)$$

B. Layer-Specific Z-Distribution

Cells are restricted to the vertical boundaries of their respective layers. For example, Layer 4 cells are exclusively placed between $-1000 \mu\text{m}$ and $-700 \mu\text{m}$. Within these boundaries, positions are assigned using a uniform random distribution to mimic the homogeneous layering observed in biological tissue.

2.4 4. Efficient Organization

- **$O(1)$ Mapping:** The function utilizes an `mtype_fast_lookup` dictionary to instantly map m-types (e.g., `L23_LBC`) to their respective cortical layers.
- **Vectorized Generation:** Using NumPy, the system generates entire populations simultaneously. This avoids iterative cell-by-cell processing, allowing the placement of tens of thousands of neurons in milliseconds.

3 Outputs

The function produces two primary data structures:

`cell_mtypes` A 1D array of length N containing the m-type label for every cell in the column.

`cell_coords` An $N \times 3$ array providing the (x, y, z) coordinates, serving as the input for subsequent distance-dependent connectivity calculations.

4 Usage Example

```
from population_engine import generate_microcolumn_cells

# Define column parameters
radius = 230 # micrometers
layer_bounds = {'L1': [0, -100], 'L23': [-100, -500]}
densities = {'L23_PC': 50000} # cells/mm3

# Generate cells
mtypes, coords = generate_microcolumn_cells(radius, layer_bounds, densities)
```